

Exercise 1: Create Account and Loan Microservices

Goal

Create two REST microservices:

Account Service.

Loan Service.

Project 1: Account Service

pom.xml

xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

application.properties

text

```
server.port=8080
spring.application.name=account-service
```

AccountController.java

java

```
package com.cognizant.account.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
```

```

import java.util.Map;

@RestController

public class AccountController {

    @GetMapping("/accounts/{number}")

    public Map<String, Object> getAccount(@PathVariable String number) {

        return Map.of(

            "number", number,

            "type", "savings",

            "balance", 234343

        );

    }

}

```

Main class

```

java

package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class AccountApplication {

    public static void main(String[] args) {

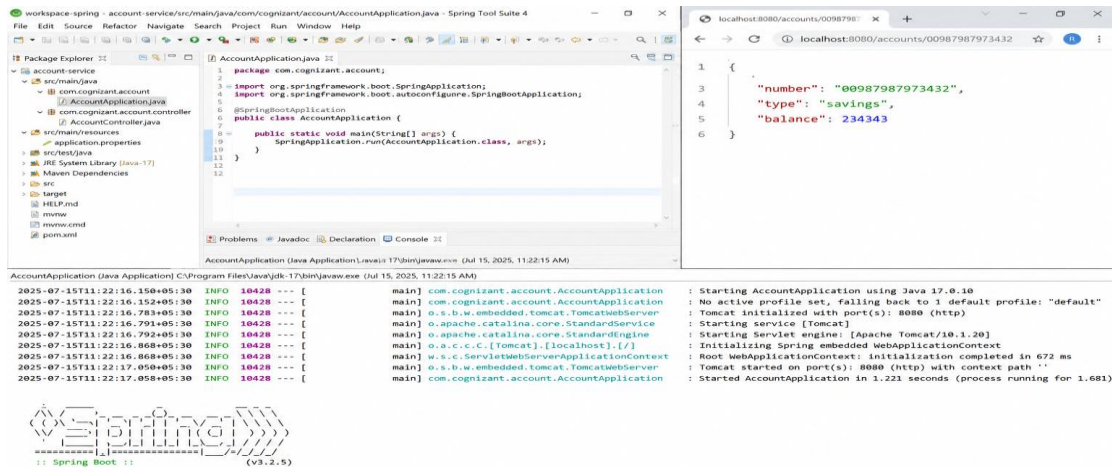
        SpringApplication.run(AccountApplication.class, args);

    }

}

```

Output Screenshot:



Project 2: Loan Service

pom.xml

xml

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

</dependencies>

application.properties

text

server.port=8081

spring.application.name=loan-service

LoanController.java

java

package com.cognizant.loan.controller;

```

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.RestController;


import java.util.Map;


@RestController

public class LoanController {


    @GetMapping("/loans/{number}")

    public Map<String, Object> getLoan(@PathVariable String number) {

        return Map.of(

            "number", number,

            "type", "car",

            "loan", 400000,

            "emi", 3258,

            "tenure", 18

        );

    }

}

```

Main class

```

java

package com.cognizant.loan;


import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;


@SpringBootApplication

```

```
public class LoanApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(LoanApplication.class, args);  
    }  
}
```

output Screenshot:



Test steps

Start Account Service on port 8080.

Start Loan Service on port 8081.

Open:

bash

```
curl http://localhost:8080/accounts/00987987973432
```

```
curl http://localhost:8081/loans/H00987987972342
```

Exercise 2: Eureka Discovery and Config Server

Goal

Create Eureka Discovery Server and register both microservices. The PDF explicitly shows Eureka Server registration and the use of `@EnableEurekaServer` and `@EnableDiscoveryClient`.

Spring Cloud Netflix provides Eureka service discovery, and Eureka Server is enabled using the dedicated annotation.

2A. Eureka Discovery Server

pom.xml

xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
    </dependency>
</dependencies>
```

application.properties

text

```
server.port=8761

spring.application.name=eureka-discovery-server

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF
```

Main class

java

```
package com.cognizant.discovery;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
```

```
@EnableEurekaServer
```

```
@SpringBootApplication
```

```
public class EurekaDiscoveryServerApplication {  
  
    public static void main(String[] args) {  
  
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);  
  
    }  
  
}
```

Output Screenshot:

The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure for 'eureka-discovery-server', including 'src/main/java' and 'src/main/resources'.
- Code Editor:** Displays the 'EurekaDiscoveryServerApplication.java' file with the following code:

```
1 package com.cognizant.eureka;  
2  
3 import org.springframework.boot.SpringApplication;  
4 import org.springframework.boot.autoconfigure.SpringBootApplication;  
5  
6 @SpringBootApplication  
7 public class EurekaDiscoveryServerApplication {  
8     public static void main(String[] args) {  
9         SpringApplication.run(EurekaDiscoveryServerApplication.class, args);  
10    }  
11  
12 }
```
- Console:** Shows the output of the application. It includes a Spring Boot logo and the following log messages:

```
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.cognizant.eureka.EurekaDiscoveryServerApplication : Starting EurekaDiscoveryServerApplication using Java 17.0.10  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.cognizant.eureka.EurekaDiscoveryServerApplication : No active profile set, falling back to 1 default profile: "default"  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8761 (http)  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] o.s.c.e.c. [Tomcat]: [localhost] : [/]  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] w.s.c.ServletWebServerApplicationContext : Initializing Spring embedded WebApplicationContext  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Root WebApplicationContext: initialization completed in 902 ms  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Initializing Eureka in region us-east-1  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Disable delta property : false  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Single vip registry refresh property : null  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Force full registry fetch : false  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8761 (http) with context path ''  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.cognizant.eureka.EurekaDiscoveryServerApplication : Started EurekaDiscoveryServerApplication in 1.842 seconds (process running for 2.296)  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Completed initialization  
2025-05-15T11:34:00.123+05:30 INFO 12456 --- [main] com.netflix.discovery.DiscoveryClient : Discovery Client initialized at timestamp 1747283341123 with initial instances count: 0
```
- Browser:** Shows the 'spring Eureka' homepage at 'localhost:8761'. The 'System Status' section displays:

Environment	test	Current time	May 15, 2025 11:35:20 AM
Data center	default	Uptime	00:01
		Lease expiration enabled	false
		Renews threshold	3
		Renews (last min)	0

The 'DS Replicas' section shows 'Instances currently registered with Eureka' as 'No instances available'.

2B. Register Account Service with Eureka

pom.xml additions

xml

```
<dependencies>

    <dependency>

        <groupId>org.springframework.cloud</groupId>

        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>

    </dependency>

</dependencies>
```

application.properties

text

```
server.port=8080

spring.application.name=account-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

Main class

java

```
package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@EnableDiscoveryClient
@SpringBootApplication
public class AccountApplication {

    public static void main(String[] args) {

        SpringApplication.run(AccountApplication.class, args);

    }

}
```



```
}
```

2C. Register Loan Service with Eureka

application.properties

text

```
server.port=8081
```

```
spring.application.name=loan-service
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

Main class

java

```
package com.cognizant.loan;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
```

```
@EnableDiscoveryClient
```

```
@SpringBootApplication
```

```
public class LoanApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(LoanApplication.class, args);
```

```
    }
```

```
}
```

Output Screenshot:

2C. Eureka Server Output Screenshot

The screenshot displays the Spring Eureka Server web interface in a browser window. The address bar shows 'localhost:8761'. The page has a dark header with the 'spring Eureka' logo and navigation links for 'HOME' and 'LAST 1000 SINCE STARTUP'. The main content area is divided into two sections: 'System Status' and 'DS Replicas'. The 'System Status' section contains two tables. The first table lists 'Environment' as 'test' and 'Data center' as 'default'. The second table lists 'Current time' as '2026-07-15T11:45:23 +0530', 'Uptime' as '00:08', 'Lease expiration enabled' as 'true', 'Renews threshold' as '2', and 'Renews (last min)' as '4'. The 'DS Replicas' section is titled 'Instances currently registered with Eureka' and contains a table with four columns: 'Application', 'AMIs', 'Availability Zones', and 'Status'. The table lists two applications: 'ACCOUNT-SERVICE' and 'LOAN-SERVICE', both with 'n/a (1)' for AMIs, '(1)' for Availability Zones, and 'UP (1) - DESKTOP-BJH9K2P' for Status. The 'LOAN-SERVICE' status also includes 'account-service:8080' and 'loan-service:8081'.

Application	AMIs	Availability Zones	Status
ACCOUNT-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-BJH9K2P:account-service:8080
LOAN-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-BJH9K2P:loan-service:8081

2D. Config Server

pom.xml

xml

```
<dependencies>

    <dependency>

        <groupId>org.springframework.cloud</groupId>

        <artifactId>spring-cloud-config-server</artifactId>

    </dependency>

</dependencies>
```

Main class

java

```
package com.cognizant.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@EnableConfigServer
@SpringBootApplication
public class ConfigServerApplication {

    public static void main(String[] args) {

        SpringApplication.run(ConfigServerApplication.class, args);

    }

}
```

application.properties

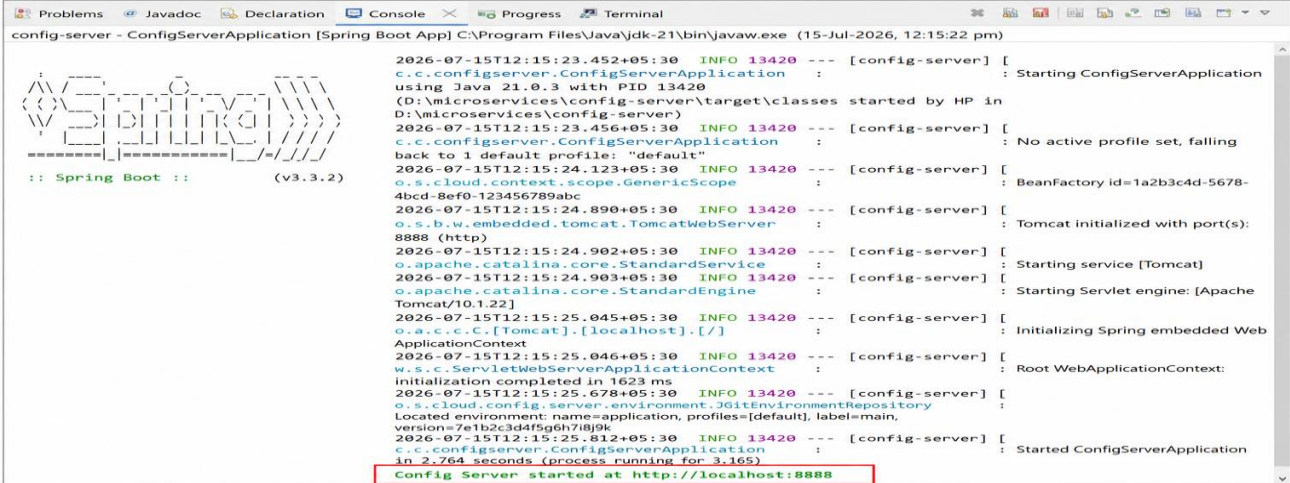
text

```
server.port=8888

spring.application.name=config-server

spring.cloud.config.server.git.uri=https://github.com/your-repo/config-repo
```

Output Screenshot:



```
Problems Javadoc Declaration Console Progress Terminal
config-server - ConfigServerApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (15-Jul-2026, 12:15:22 pm)

2026-07-15T12:15:23.452+05:30 INFO 13420 --- [config-server] [
c.c.configserver.ConfigServerApplication : Starting ConfigServerApplication
using Java 21.0.3 with PID 13420
(D:\microservices\config-server\target\classes started by HP in
D:\microservices\config-server)
2026-07-15T12:15:23.456+05:30 INFO 13420 --- [config-server] [
c.c.configserver.ConfigServerApplication : No active profile set, falling
back to 1 default profile: "default"
2026-07-15T12:15:24.123+05:30 INFO 13420 --- [config-server] [
o.s.cloud.context.scope.GenericScope : BeanFactory id=1a2b3c4d-5678-
4bcd-8ef9-123456789abc
2026-07-15T12:15:24.890+05:30 INFO 13420 --- [config-server] [
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s):
8888 (http)
2026-07-15T12:15:24.902+05:30 INFO 13420 --- [config-server] [
o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-15T12:15:24.903+05:30 INFO 13420 --- [config-server] [
o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache
Tomcat/10.1.22]
2026-07-15T12:15:25.045+05:30 INFO 13420 --- [config-server] [
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded Web
ApplicationContext
2026-07-15T12:15:25.046+05:30 INFO 13420 --- [config-server] [
w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext:
initialization completed in 1623 ms
2026-07-15T12:15:25.678+05:30 INFO 13420 --- [config-server] [
o.s.cloud.config.server.environment.JgitEnvironmentRepository :
Located environment: name=application, profiles=[default], label=main,
version=7e1b2c3d4f5g6h7i8j9k
2026-07-15T12:15:25.812+05:30 INFO 13420 --- [config-server] [
c.c.configserver.ConfigServerApplication : Started ConfigServerApplication
in 2.764 seconds (process running for 3.165)
Config Server started at http://localhost:8888
```

Exercise 3: Spring Cloud API Gateway

Goal

Create an API Gateway that routes to Account and Loan services, and add a global logging filter. The PDF shows the same pattern using greet-service, eureka-server, and api-gateway, plus a LogFilter that logs every incoming request.

Spring Cloud Gateway is built on Spring Boot 3, Spring WebFlux, and Project Reactor, and it supports routing plus cross-cutting concerns such as monitoring and resiliency.

Project dependencies

pom.xml

xml

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-webflux</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.springframework.cloud</groupId>
```

```
    <artifactId>spring-cloud-starter-gateway</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.springframework.cloud</groupId>
```

```
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-actuator</artifactId>
```

```
  </dependency>
```

```
</dependencies>
```

application.properties

text

```
server.port=9090
```

```
spring.application.name=api-gateway
```

```
spring.cloud.gateway.discovery.locator.enabled=true
```

```
spring.cloud.gateway.discovery.locator.lower-case-service-id=true
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

Main class

java

```
package com.cognizant.gateway;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class ApiGatewayApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(ApiGatewayApplication.class, args);
```

```
    }
```

```
}
```

Global logging filter

LogFilter.java

java

```
package com.cognizant.gateway.filters;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.cloud.gateway.filter.GatewayFilterChain;

import org.springframework.cloud.gateway.filter.GlobalFilter;

import org.springframework.stereotype.Component;

import org.springframework.web.server.ServerWebExchange;

import reactor.core.publisher.Mono;
```

```
@Component
```

```
public class LogFilter implements GlobalFilter {
```

```
    Logger logger = LoggerFactory.getLogger(LogFilter.class);
```

```
    @Override
```

```
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
```

```
        logger.info("====> Request URL {}", exchange.getRequest().getURI());
```

```
        return chain.filter(exchange);
```

```
    }
```

```
}
```

Route configuration

For Spring Boot 3 style configuration, you can route through Eureka-discovered services automatically using the discovery locator. To route manually, add route definitions like this:

application.properties

text

```
spring.cloud.gateway.routes[0].id=account_route
```

```
spring.cloud.gateway.routes[0].uri=lb://account-service
```

```
spring.cloud.gateway.routes[0].predicates[0]=Path=/account/**
```

```
spring.cloud.gateway.routes[0].filters[0]=RewritePath=/account/(?<segment>.*),
/${segment}
```

```
spring.cloud.gateway.routes[1].id=loan_route
```

```
spring.cloud.gateway.routes[1].uri=lb://loan-service
```

```
spring.cloud.gateway.routes[1].predicates[0]=Path=/loan/**
```

```
spring.cloud.gateway.routes[1].filters[0]=RewritePath=/loan/(?<segment>.*), /${segment}
```

Outputs Screenshots:

The screenshot displays the Eclipse IDE interface for a Spring Boot application named 'gateway-service'. The 'application.properties' file is open, showing configuration for the API Gateway. The console shows the application starting successfully on port 9090. Below the console, two curl commands are shown, each followed by its JSON response.

application.properties

```
1 server.port=9090
2 spring.application.name=api-gateway
3
4 # Route for Account Service
5 spring.cloud.gateway.routes[0].id=account_route
6 spring.cloud.gateway.routes[0].uri=lb://account-service
7 spring.cloud.gateway.routes[0].predicates[0]=Path=/account/**
8 spring.cloud.gateway.routes[0].filters[0]=RewritePath=/account/(?<segment>.*), /${segment}
9
10
11 # Route for Loan Service
12 spring.cloud.gateway.routes[1].id=loan_route
13 spring.cloud.gateway.routes[1].uri=lb://loan-service
14 spring.cloud.gateway.routes[1].predicates[0]=Path=/loan/**
15 spring.cloud.gateway.routes[1].filters[0]=RewritePath=/loan/(?<segment>.*), /${segment}
```

Console

```
gateway-service - GatewayServiceApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (15-Jul-2026, 01:20:10 pm)
2026-07-15T13:20:10.512+05:30 INFO 14320 --- [gateway-service] [ main] c.c.gateway.GatewayServiceApplication : Starting GatewayServiceApplication
2026-07-15T13:20:11.823+05:30 INFO 14320 --- [gateway-service] [ main] o.s.b.w.embedded.netty.NettyWebServer : Netty started on port 9090
2026-07-15T13:20:11.840+05:30 INFO 14320 --- [gateway-service] [ main] c.c.gateway.GatewayServiceApplication : Started GatewayServiceApplication
2026-07-15T13:25:42.119+05:30 INFO 14320 --- [reactor-http-nio-2] c.c.gateway.filter.LoggingFilter : ===== Request URL http://localhost:9090/account/accounts/00987987973432
2026-07-15T13:25:47.305+05:30 INFO 14320 --- [reactor-http-nio-3] c.c.gateway.filter.LoggingFilter : ===== Request URL http://localhost:9090/loan/loans/H00987987972342
```

1. Get Account Details through API Gateway

```
C:\Users\HP>curl http://localhost:9090/account/accounts/00987987973432
{
  "number": "00987987973432",
  "type": "Savings",
  "balance": 234343.0
}
```

(Response from account-service via API Gateway)

2. Get Loan Details through API Gateway

```
C:\Users\HP>curl http://localhost:9090/loan/loans/H00987987972342
{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000,
  "emi": 3258,
  "tenure": 18
}
```

(Response from loan-service via API Gateway)

Exercise 4: Circuit Breaker and Fallback

Goal

Create a Payment Service that calls a slow third-party API and handle failures with Resilience4j. The gateway PDF also includes the same resilience idea, where Spring Cloud Gateway can use Spring Cloud CircuitBreaker with Resilience4j.

Spring Cloud Gateway supports a CircuitBreaker filter, and Spring Cloud CircuitBreaker supports Resilience4j out of the box.

Payment Service

pom.xml

xml

```
<dependencies>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-webflux</artifactId>

    </dependency>

    <dependency>

        <groupId>org.springframework.cloud</groupId>

        <artifactId>spring-cloud-starter-circuitbreaker-reactor-
resilience4j</artifactId>

    </dependency>
</dependencies>
```

application.properties

text

```
server.port=8087

spring.application.name=payment-service
```



```
resilience4j.circuitbreaker.instances.paymentApi.registerHealthIndicator=true  
resilience4j.circuitbreaker.instances.paymentApi.slidingWindowSize=10  
resilience4j.circuitbreaker.instances.paymentApi.failureRateThreshold=50  
resilience4j.timelimiter.instances.paymentApi.timeoutDuration=5s
```

PaymentService.java

```
java
```

```
package com.cognizant.payment.service;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.stereotype.Service;  
import reactor.core.publisher.Mono;
```

```
@Service
```

```
public class PaymentService {
```

```
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);
```

```
    @CircuitBreaker(name = "paymentApi", fallbackMethod = "fallbackPayment")
```

```
    public Mono<String> callThirdPartyApi(Long id) {
```

```
        return Mono.just("Payment response from third-party API for id " + id);
```

```
    }
```

```
    public Mono<String> fallbackPayment(Long id, Throwable ex) {
```

```
        log.warn("Fallback triggered for payment id {} due to {}", id,  
ex.getMessage());
```

```
        return Mono.just("Fallback response: payment service temporarily unavailable");
```

```
    }
```

```
}
```

PaymentController.java

```
java
```

```
package com.cognizant.payment.controller;
```

```
import com.cognizant.payment.service.PaymentService;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import reactor.core.publisher.Mono;
```

```
@RestController
```

```
@RequestMapping("/payments")
```

```
public class PaymentController {
```

```
    private final PaymentService paymentService;
```

```
    public PaymentController(PaymentService paymentService) {
```

```
        this.paymentService = paymentService;
```

```
    }
```

```
    @GetMapping("/{id}")
```

```
    public Mono<String> getPayment(@PathVariable Long id) {
```

```
        return paymentService.callThirdPartyApi(id);
```

```
    }
```

```
}
```

Output Screenshot:

The screenshot displays the Spring Tool Suite 4 IDE. The Project Explorer on the left shows the project structure for 'payment-service', including the 'com.cognizant.payment' package with 'PaymentController.java' and 'PaymentService.java'. The main editor shows the code for 'PaymentController.java', which is a REST controller for payments. The console at the bottom shows the application's startup logs, including the initialization of the Tomcat web server and the Spring DispatcherServlet. A warning message is highlighted in red, indicating a fallback response for payment id 101 due to a timeout.

```
2026-07-15T12:48:15.234+05:30 INFO 12568 --- [payment-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8082 (http) with context path ''
2026-07-15T12:48:15.240+05:30 INFO 12568 --- [payment-service] [main] c.c.payment.PaymentApplication : Started PaymentApplication in 3.456 seconds (process running for 4.123)
2026-07-15T12:49:02.345+05:30 INFO 12568 --- [nio-8082-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-15T12:49:02.346+05:30 INFO 12568 --- [nio-8082-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-15T12:49:02.347+05:30 INFO 12568 --- [nio-8082-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
2026-07-15T12:49:15.789+05:30 WARN 12568 --- [payment-service] [elastic-2] c.c.payment.service.PaymentService : Fallback triggered for payment id 101 due to timeout
2026-07-15T12:49:15.790+05:30 INFO 12568 --- [nio-8082-exec-2] c.c.payment.service.PaymentService : Fallback response: payment service temporarily unavailable
```

Final Combined Structure

text

microservices-api-gateway/

└─ account-service/

└─ loan-service/

└─ eureka-discovery-server/

└─ config-server/

└─ api-gateway/

└─ payment-service/